

## Imperative vs Declarative in Kubernetes

These are **two different ways** to create and manage Kubernetes resources.

---

### 1. Imperative Approach

#### Simple Definition

**Imperative means you tell Kubernetes exactly HOW to do something using commands.**

You give step-by-step instructions.

#### Real-Life Example 🍳

Imagine cooking noodles.

You tell someone:

1. Boil water.
2. Add noodles.
3. Cook for 2 minutes.
4. Add seasoning.
5. Serve.

You are explaining **every step**.

This is **Imperative**.

---

### Kubernetes Example

Create a Deployment directly:

```
kubectl create deployment nginx --image=nginx
```

Kubernetes immediately creates the Deployment.

You told Kubernetes exactly what command to run.

---

### Characteristics

- Uses **kubectl** commands

- Fast and simple
  - Good for testing and learning
  - No YAML file required
  - Changes are not saved in a configuration file
- 

## Common Imperative Commands

Create Deployment

```
kubectl create deployment nginx --image=nginx
```

Scale Deployment

```
kubectl scale deployment nginx --replicas=3
```

Delete Deployment

```
kubectl delete deployment nginx
```

---

## 2. Declarative Approach

### Simple Definition

**Declarative means you tell Kubernetes WHAT you want, and Kubernetes figures out HOW to achieve it.**

You describe the desired state.

---

### Real-Life Example 🏠

You tell a builder:

"I want a house with 3 bedrooms and 2 bathrooms."

You **don't tell** them:

- how to lay bricks
- how to build walls
- how to install windows

The builder decides **how** to build it.

This is **Declarative**.

---

### Kubernetes Example

Create a YAML file named deployment.yaml

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  name: nginx
```

```
spec:
```

```
  replicas: 3
```

```
  selector:
```

```
    matchLabels:
```

```
      app: nginx
```

```
  template:
```

```
    metadata:
```

```
      labels:
```

```
        app: nginx
```

```
  spec:
```

```
    containers:
```

```
      - name: nginx
```

```
image: nginx
```

Apply it:

```
kubectl apply -f deployment.yaml
```

Kubernetes reads the YAML file and creates the Deployment.

---

### Characteristics

- Uses **YAML files**
  - Describes the desired state
  - Best for production
  - Easy to update
  - Easy to store in GitHub
  - Supports version control
- 

### Real-Life Comparison

#### Imperative



Tell the chef every cooking step.

1. Cut onions.
  2. Heat oil.
  3. Fry onions.
  4. Add spices.
- 

#### Declarative



Simply say:

"I want chicken biryani."

The chef knows how to prepare it.

---

## Kubernetes Comparison

### Imperative

You

|



kubect! create deployment

|



Kubernetes

You tell Kubernetes **how** to do it.

---

### Declarative

You

|



deployment.yaml

|



kubect! apply -f deployment.yaml

|



Kubernetes

You tell Kubernetes **what** you want.

---

## Advantages

### Imperative

- ✓ Simple
  - ✓ Quick
  - ✓ Good for beginners
  - ✓ Useful for testing
- 

### **Declarative**

- ✓ Best for production
  - ✓ Easy to update
  - ✓ Easy rollback
  - ✓ Can store YAML files in Git
  - ✓ Team collaboration
- 

### **Disadvantages**

#### **Imperative**

- ✗ Difficult to track changes
  - ✗ No configuration file
  - ✗ Not ideal for large projects
- 

#### **Declarative**

- ✗ Requires writing YAML files
  - ✗ Slightly more complex for beginners
- 

### **Example: Create 3 Nginx Pods**

#### **Imperative**

kubectl create deployment nginx --image=nginx

```
kubectl scale deployment nginx --replicas=3
```

---

## Declarative

Create a YAML file:

```
replicas: 3
```

```
image: nginx
```

Run:

```
kubectl apply -f deployment.yaml
```

---

## Imperative vs Declarative

| Feature            | Imperative                 | Declarative                 |
|--------------------|----------------------------|-----------------------------|
| Meaning            | Tell Kubernetes <b>how</b> | Tell Kubernetes <b>what</b> |
| Uses               | Commands                   | YAML files                  |
| Configuration File | ❌ No                       | ✅ Yes                       |
| Version Control    | ❌ Difficult                | ✅ Easy                      |
| Production Use     | ❌ Rare                     | ✅ Recommended               |
| Best For           | Learning & testing         | Production environments     |
| Command            | kubectl create             | kubectl apply -f            |

---

## Most Common Commands

### Imperative

```
kubectl create deployment nginx --image=nginx
```

```
kubectl scale deployment nginx --replicas=3
```

```
kubectl delete deployment nginx
```

---

## Declarative

Apply configuration:

```
kubectl apply -f deployment.yaml
```

View resources:

```
kubectl get deployments
```

Delete using the YAML file:

```
kubectl delete -f deployment.yaml
```

---

## Easy Memory Trick

Imperative

|



Tell HOW

(Command Based)

```
kubectl create
```

```
kubectl run
```

```
kubectl delete
```

-----

Declarative

|



Tell WHAT

(YAML Based)



```
kubectl apply -f file.yaml
```

---

## KCNA Interview Answer

### What is Imperative?

Imperative is a command-based approach where you tell Kubernetes exactly how to perform an action using kubectl commands.

### What is Declarative?

Declarative is a configuration-based approach where you define the desired state in a YAML file, and Kubernetes ensures that state is maintained.

---

## One-Line Memory Trick

-  Imperative = Commands = Tell Kubernetes HOW.
-  Declarative = YAML Files = Tell Kubernetes WHAT.

For the KCNA exam: Learn both approaches, but remember that **Declarative (YAML + kubectl apply)** is the recommended approach for real-world Kubernetes deployments.